

TransitFlow Reflection & Trade-offs說明

Section 6 — Reflection & Trade-offs

Trade-off 1: Primary Key Strategy in PostgreSQL (UUID/SERIAL vs. Natural Keys)

在設計關聯式資料庫時，我們面臨了主鍵選擇的抉擇。原始的 JSON 測資提供了如 `U001`、`MS_SCH01` 等字串代碼。

- 我們的決策：我們決定捨棄將這些字串作為 PK。對於主檔參考表 (如車站、時刻表) 我們採用 `SERIAL` 整數；對於交易表 (如訂票、使用者、刷卡紀錄) 我們採用 `UUID`。原有的 JSON 字串代碼則被存為 `UNIQUE` 的 `_code` 欄位以利 Seeding 對應。
- **Trade-off**：我們犧牲了資料庫直接查詢時的「人類可讀性」，且 `UUID` 比一般整數佔用更多儲存空間。但這個妥協換來了極大的好處：`UUID` 避免了在公開 API 或前端暴露連續的訂單編號 (防止競爭對手推算營運量)，同時 `SERIAL` 讓關聯表的 JOIN 效能大幅提升，徹底將業務邏輯代碼與資料庫底層身分解耦。

Trade-off 2: Real-time Disruption Modeling in Neo4j

在實作即時故障避障功能時，我們必須決定如何處理車站關閉的狀態。

- 我們的決策：我們選擇不從資料庫中刪除節點或連線，也不在 Python 應用層做過濾。相反地，我們將 `closed` 狀態作為一個布林屬性 (`closed: false`) 直接存放在 Neo4j 的 Node 上。
- **Trade-off**：我們犧牲了每一次 Cypher 查詢的極致效能，因為每次尋徑都必須額外執行 `WHERE NONE(n IN nodes(path) WHERE n.closed = true)` 來動態檢查節點狀態。然而，這個微小的運算成本，換來了維持圖形拓撲絕對不變的巨大優勢。我們可以在不重建圖形結構的情況下，瞬間開關車站狀態，完美實現即時的延誤連鎖分析與動態繞道演算法。

Production Improvement: Scaling for a Metropolis

目前的 TransitFlow 系統架構在測試環境下運行良好，但若要部署至真實世界，我們會針對資料讀寫瓶頸進行以下架構升級：

- 導入 **Redis** 快取層與讀寫分離：目前 Agent 的每一次查詢 (包含查詢時刻表、票價、空位) 都會直接敲擊 PostgreSQL 或 Neo4j 底層資料庫。在真實情境中，交通系統的特性是讀取量極大但時刻表更新頻率低。我們會在架構中加入 Redis 記憶體快取來攔截 90% 以上的單純查詢請求。此外，我們會將 PostgreSQL 設定為讀寫分離，讓大量的查詢請求分流到唯讀資料庫，確保核心的寫入節點 (Master DB) 能專心處理 `make_booking` 與 `cancel_booking` 的 ACID 併發交易，避免高流量導致系統死鎖或崩潰。